

Week 1: Logic Initialization & Requirements Gathering

Implementation of Core Features: We established one of the main features of the app. Using the datetime library, we created a backend script capable of tracking time, ensuring the application could mathematically track user deadlines.

Technologies Used:

- Python datetime: Chosen for its native ability to handle time-zone-aware calculations without external dependencies.
- Design Decisions: We chose to prioritize the back-end first. By securing the time-sensitive data logic before the visuals, we ensured the application's core functionality was robust.

Ethical Considerations: During the research for SQL databases, we discussed User Privacy. Since the app handles personal schedules, we decided that any future database implementation must ensure data is stored locally to prevent unauthorized access to a user's daily routine.

Week 2: GUI Prototyping & System Debugging

Implementation of Core Features: We started working on the front-end of the app. By drafting an Interface Mockup, we mapped out where buttons, labels, and entry fields would sit.

Technologies Used:

- Tkinter: Selected because it is lightweight and comes pre-installed with Python, making it ideal for rapid GUI prototyping without complex environment setups.

Backend–Frontend Communication: We began "hooking" the Week 1 datetime logic into the GUI buttons. For example, when the user wants to set a time for a requirement, the system will allow the user to set a time and set notifications accordingly when those features are implemented in later stages.

Trade-offs: We chose simplicity over aesthetics. While other libraries (like PyQt) offer more modern looks, Tkinter allowed us to fix Week 1 bugs faster due to its straightforward documentation.

Week 3: Data Architecture & Variable Mapping

Implementation of Core Features: We moved from "static" displays to "dynamic" data handling. Through Variable Mapping, we ensured that text typed into a Tkinter entry box was correctly captured and stored as a Python variable.

Technologies Used:

- Dictionaries & Lists: Used for temporary data storage. Dictionaries were preferred for user profiles (Key: UserID, Value: Info), while Lists were used for chronological task entries.
- Key Design Decisions: We opted for In-memory Storage (Dictionaries) as a middle step before moving to a permanent database. This allowed us to test the "Save" and "Edit" functions without the complexity of SQL queries yet.

Teamwork & Consistency: To maintain code integrity, we used a "Pair Programming" style, ensuring that the frontend variables matched the backend expectations to avoid "KeyErrors" during runtime.

Week 4: Performance Optimization & Feature Scoping

Implementation of Core Features: The "Information System" (how the app remembers things) reached 80% completion. We also began working on a notification system to alert users of upcoming tasks.

Technologies Used:

- numpy: Integrated to handle data processing more efficiently. While simple lists work for small schedules, numpy arrays provide faster indexing and better performance as the user's data grows.

Backend–Frontend Communication: We refined the bridge between the UI and the data storage, ensuring that when a user closes the app, their information is correctly serialized and prepared for persistent storage.

Ethical Considerations (Accessibility): During GUI Polishing, we focused on Visual Stability. We made a conscious decision to use high-contrast colors and clear fonts to ensure the interface is accessible to users with visual impairments.

